Hoare Logic

A 90-minute PhD Course Lecture

PhD Course

May 18, 2026

Why Hoare Logic?

- We want mathematical evidence that a program satisfies a contract.
- Tests show particular executions; Hoare logic reasons about all states.
- The key idea is local: prove small commands correct, compose proofs.

$$\{P\}\ c\ \{Q\}$$

If command c starts in a state satisfying P, it ends in a state satisfying Q.

Running Example: Integer Division by Subtraction

```
q := 0;
r := x;
while y <= r do
  r := r - y;
  q := q + 1
```

Assume $x \geq 0$ and $y > 0$. Desired contract:

$$\{x \geq 0 \wedge y > 0\}\ \texttt{Div}\ \{x = q \cdot y + r \wedge 0 \leq r < y\}$$

IMP: A Tiny Imperative Language

$$
\begin{array}{lcl}
a & ::= & n \mid x \mid a_1 + a_2 \mid a_1 - a_2 \mid a_1 \cdot a_2 \\
b & ::= & \text{true} \mid \text{false} \mid a_1 = a_2 \mid a_1 \leq a_2 \mid \neg b \mid b_1 \wedge b_2 \\
c & ::= & \text{skip} \mid x := a \mid c_1; c_2 \mid \text{if } b \text{ then } c_1 \text{ else } c_2 \mid \text{while } b \text{ do } c
\end{array}
$$

A state $\sigma \in \Sigma$ maps variables to integers.

$$
\sigma[x \mapsto v](z) = \begin{cases} v & z = x \\ \sigma(z) & z \neq x. \end{cases}
$$

Big-Step Semantics for IMP

$$\langle c, \sigma \rangle \Downarrow \sigma'$$

means command c, started in state σ, terminates in state σ'.

$$\frac{}{\langle \mathsf{skip}, \sigma \rangle \Downarrow \sigma} \qquad \frac{}{\langle x := a, \sigma \rangle \Downarrow \sigma[x \mapsto [\![a]\!]_\sigma]}$$

$$\frac{\langle c_1, \sigma \rangle \Downarrow \sigma' \quad \langle c_2, \sigma' \rangle \Downarrow \sigma''}{\langle c_1; c_2, \sigma \rangle \Downarrow \sigma''}$$

$$\frac{[\![b]\!]_\sigma = \mathsf{true} \quad \langle c_1, \sigma \rangle \Downarrow \sigma'}{\langle \mathsf{if}\ b\ \mathsf{then}\ c_1\ \mathsf{else}\ c_2, \sigma \rangle \Downarrow \sigma'} \qquad \frac{[\![b]\!]_\sigma = \mathsf{false} \quad \langle c_2, \sigma \rangle \Downarrow \sigma'}{\langle \mathsf{if}\ b\ \mathsf{then}\ c_1\ \mathsf{else}\ c_2, \sigma \rangle \Downarrow \sigma'}$$

Big-Step Semantics for While

$$\frac{[\![b]\!]_\sigma = \text{false}}{\langle \text{while } b \text{ do } c, \sigma\rangle \Downarrow \sigma}$$

$$\frac{[\![b]\!]_\sigma = \text{true} \quad \langle c, \sigma\rangle \Downarrow \sigma' \quad \langle \text{while } b \text{ do } c, \sigma'\rangle \Downarrow \sigma''}{\langle \text{while } b \text{ do } c, \sigma\rangle \Downarrow \sigma''}$$

- Big-step semantics naturally describes terminating executions. Nontermination is represented by absence of a final state.
- This semantics is deterministic

Assertions and Valid Triples

An assertion is a predicate over states:

$$P, Q : \Sigma \to \{\text{true}, \text{false}\}.$$

Partial correctness:

$$\models_{\text{par}} \{P\}\ c\ \{Q\} \quad \text{iff} \quad \forall \sigma, \sigma'.\ P(\sigma) \wedge \langle c, \sigma \rangle \Downarrow \sigma' \Rightarrow Q(\sigma').$$

It says: if c terminates, the answer is right.

Partial vs. Total Correctness

Partial correctness:

$$\models_{\text{par}} \{P\}\ c\ \{Q\} \quad \text{iff} \quad \forall \sigma, \sigma'.\ P(\sigma) \wedge \langle c, \sigma \rangle \Downarrow \sigma' \Rightarrow Q(\sigma').$$

Total correctness:

$$\models_{\text{tot}} \{P\}\ c\ \{Q\} \quad \text{iff} \quad \forall \sigma.\ P(\sigma) \Rightarrow \exists \sigma'.\ \langle c, \sigma \rangle \Downarrow \sigma' \wedge Q(\sigma').$$

Classic example:

$$\{\text{true}\}\ \text{while true do skip}\ \{\text{false}\}$$

is valid for partial correctness, but invalid for total correctness.

Direct Semantic Proof: A Small Program

Program:

```
x := x + 1;
y := x
```

$$\models_{\text{par}} \{x = n \wedge 0 < n \wedge n < \text{max\_int} + 1\}\ x := x + 1; y := x\ \{x > 0 \wedge y > 0\}.$$

Proof from IMP semantics:

1. Let $\sigma(x) = n$ with $0 < n < \text{max\_int} + 1$, and suppose the program terminates in σ''.
2. Assignment gives $\langle x := x + 1, \sigma \rangle \Downarrow \sigma'$, where $\sigma' = \sigma[x \mapsto n + 1]$.
3. Sequencing gives $\langle y := x, \sigma' \rangle \Downarrow \sigma''$, where $\sigma'' = \sigma'[y \mapsto \sigma'(x)]$.
4. Since $\sigma'(x) = n + 1$, we have $\sigma''(y) = n + 1$ and $\sigma''(x) = n + 1$, so both are > 0. The bound $n < \text{max\_int} + 1$ keeps the increment within range.

Hoare Logic as a Proof System

Semantic validity:

$$\models \{P\}\ c\ \{Q\}$$

means the triple is true in the operational semantics.

Syntactic derivability:

$$\vdash \{P\}\ c\ \{Q\}$$

means the triple can be derived from proof rules.

The central meta-theoretic questions:

- Soundness: if $\vdash \{P\}\ c\ \{Q\}$, then $\models \{P\}\ c\ \{Q\}$.
- Completeness: if $\models \{P\}\ c\ \{Q\}$, then $\vdash \{P\}\ c\ \{Q\}$.

Rules for Skip, Assignment, and Sequencing

$$\frac{}{\{P\}\ \text{skip}\ \{P\}}\ \text{Skip}$$

$$\frac{}{\{Q[x \mapsto a]\}\ x := a\ \{Q\}}\ \text{Assign}$$

$$Q[x \mapsto a]$$

is capture-avoiding substitution of expression a for free occurrences of x in Q.

Without capture-avoidance, substituting $x \mapsto y + 1$ in

$$\forall y.\ x < y$$

would wrongly give $\forall y.\ y + 1 < y$, because the free y gets captured. Renaming first yields the correct result:

$$\forall z.\ y + 1 < z.$$

Assignment Works Backwards

To prove:

$$\{?\}\ x := x + 1\ \{x > 5\}$$

use the assignment rule:

$$\{x + 1 > 5\}\ x := x + 1\ \{x > 5\}.$$

So a natural precondition is:

$$x > 4.$$

Common pitfall: the postcondition talks about the state *after* the assignment. So the claim

$$\{x > 5\}\ x := x + 1\ \{x > 6\}$$

is valid, but you do *not* prove it by checking $x > 6$ in the pre-state. The right backward step is to replace the final x by $x + 1$, giving the precondition $x + 1 > 6$, i.e. $x > 5$. The confusion is between the old value of x and the new value assigned by the command.

Sequential composition and consequence

$$\frac{\{P\}\ c_1\ \{Q\} \quad \{Q\}\ c_2\ \{R\}}{\{P\}\ c_1; c_2\ \{R\}}\ \text{Seq}$$

$$\frac{P' \Rightarrow P \quad \{P\}\ c\ \{Q\} \quad Q \Rightarrow Q'}{\{P'\}\ c\ \{Q'\}}\ \text{Consequence}$$

Consequence is where mathematical reasoning about assertions enters the proof. The program logic delegates arithmetic, algebra, and domain facts to the assertion logic.

Example, for the program on slide 9:

$$\{x = n \wedge 0 < n \wedge n < \text{max\_int} + 1\}\ x := x + 1; y := x\ \{x > 0 \wedge y > 0\}.$$

The derivation is:

- $\{x = n\}\ x := x + 1\ \{x = n + 1\}$ by Assign.
- $\{x = n + 1\}\ y := x\ \{x = n + 1 \wedge y = n + 1\}$ by Assign.
- $\{x = n\}\ x := x + 1; y := x\ \{x = n + 1 \wedge y = n + 1\}$ by Seq.
- $x = n \wedge 0 < n \wedge n < \text{max\_int} + 1 \Rightarrow x = n + 1 \wedge y = n + 1 \Rightarrow x > 0 \wedge y > 0$.
- Therefore $\{x = n \wedge 0 < n \wedge n < \text{max\_int} + 1\}\ x := x + 1; y := x\ \{x > 0 \wedge y > 0\}$ by Consequence.

Rules for choice

$$\frac{\{P \wedge b\}\ c_1\ \{Q\} \quad \{P \wedge \neg b\}\ c_2\ \{Q\}}{\{P\}\ \text{if}\ b\ \text{then}\ c_1\ \text{else}\ c_2\ \{Q\}}\ \text{If}$$

Example: compute the maximum of two numbers.

```
if x <= y then
  m := y
else
  m := x
```

$$\{\text{true}\}\ \texttt{max}\ \{m \geq x \wedge m \geq y \wedge (m = x \vee m = y)\}.$$

Max: Derivation

Let

$$Q \equiv m \geq x \wedge m \geq y \wedge (m = x \vee m = y).$$

$$\{\text{true}\} \text{ if } x \leq y \text{ then } m := y \text{ else } m := x \; \{Q\}$$

$$\text{If} \quad \begin{array}{l} \{x \leq y\} \; m := y \; \{Q\} \\ \{x > y\} \; m := x \; \{Q\} \end{array}$$

Left branch:

$$Q[y/m] \equiv y \geq x \wedge y \geq y \wedge (y = x \vee y = y)$$

$$\{Q[y/m]\} \; m := y \; \{Q\} \qquad \text{Assign}$$

$$x \leq y \Rightarrow Q[y/m] \qquad \text{Consequence}$$

Right branch:

$$Q[x/m] \equiv x \geq x \wedge x \geq y \wedge (x = x \vee x = y)$$

$$\{Q[x/m]\} \; m := x \; \{Q\} \qquad \text{Assign}$$

$$x > y \Rightarrow Q[x/m] \qquad \text{Consequence}$$

Rule for While

$$\frac{\{I \wedge b\}\ c\ \{I\}}{\{I\}\ \text{while } b \text{ do } c\ \{I \wedge \neg b\}}\ \text{While}$$

I is a loop invariant:

- Initialization: I holds before the loop.
- Preservation: one loop body iteration preserves I.
- Exit: $I \wedge \neg b$ is strong enough to imply the desired postcondition.
- Intuition: An inductive hypothesis for programs

Rule Use on the Running Example

For

```
while y <= r do
  r := r - y;
  q := q + 1
```

take:

$$I \equiv x = q \cdot y + r \wedge r \geq 0 \wedge y > 0.$$

Backward reasoning for the loop body:

$$\{I \wedge y \leq r\}\ r := r - y;\ q := q + 1\ \{I\}.$$

$\{x = (q+1) \cdot y + r \wedge r \geq 0 \wedge y > 0\}\ q := q + 1\ \{I\}$ Assign
$\{x = (q+1) \cdot y + (r-y) \wedge r - y \geq 0 \wedge y > 0\}\ r := r - y\ \{x = (q+1) \cdot y + r \wedge r \geq 0 \wedge y > 0\}$ Assign
$\{x = (q+1) \cdot y + (r-y) \wedge r - y \geq 0 \wedge y > 0\}\ r := r - y;\ q := q + 1\ \{I\}$ Seq
$\{x = q \cdot y + r \wedge r - y \geq 0 \wedge y > 0\}\ r := r - y;\ q := q + 1\ \{I\}$ Consequence
$\{I \wedge y \leq r\}\ r := r - y;\ q := q + 1\ \{I\}$ Consequence

Exercise 1: Assignment Precondition

Find a weakest-looking precondition P:

$$\{P\}\ x := x + 2\ \{x \geq 10\}.$$

Then prove the following by the assignment rule and consequence:

$$\{x \geq 8\}\ x := x + 2\ \{x \geq 10\}.$$

- Why is $x \geq 10$ too strong but still valid?
- Why is $x \geq 7$ too weak?

Exercise 2: Conditional

Prove:

$$\{\text{true}\}\ \text{if } x < 0 \text{ then } y := -x \text{ else } y := x\ \{y \geq 0\}$$

Exercise 3: Division Initialization

For the running example:

```
q := 0;
r := x
```

Show:

$$\{x \geq 0 \wedge y > 0\}\ q := 0;\ r := x\ \{I\}$$

where

$$I \equiv x = q \cdot y + r \wedge r \geq 0 \wedge y > 0.$$

Exercise 5: Find the Invariant

Program:

$$s := 0;\ \ i := 0;\ \text{while } i < n \text{ do } (i := i + 1;\ \ s := s + i)$$

Desired postcondition:

$$s = \frac{n(n+1)}{2}.$$

Find an invariant and prove the contract

Loops Need Two Ideas

Partial correctness needs an invariant:

$$\{I \wedge b\}\ c\ \{I\} \quad \Rightarrow \quad \{I\}\ \text{while}\ b\ \text{do}\ c\ \{I \wedge \neg b\}.$$

Total correctness also needs termination.

A variant/ranking function $V : \Sigma \to \mathbb{N}$ decreases on every iteration:

$$I \wedge b \Rightarrow V \geq 0 \qquad \{I \wedge b \wedge V = v\}\ c\ \{I \wedge V < v\}.$$

Choosing Invariants

Useful heuristics:

- Start from the postcondition and weaken what is not yet true.
- Add bounds needed for exit reasoning and termination.
- Include preservation facts: constants, conservation laws, algebraic relations.
- For counters, relate accumulated state to the current counter value.

For division:

$$x = q \cdot y + r \wedge r \geq 0 \wedge y > 0$$

is the preserved relationship.

Variant for Division

Loop guard:

$$y \leq r.$$

Variant:

$$V = r.$$

Under the invariant and guard:

$$r \geq 0, \qquad y > 0, \qquad r' = r - y < r.$$

Therefore r decreases over $\mathbb{N}$, so the loop terminates.

Partial Correctness of Division

From the invariant and exit condition:

$$I \wedge \neg(y \leq r)$$

we get:

$$x = q \cdot y + r \wedge r \geq 0 \wedge y > 0 \wedge r < y.$$

Hence:

$$x = q \cdot y + r \wedge 0 \leq r < y.$$

Total correctness follows by adding the variant r.

Well-Founded Relations

Termination arguments need a notion of "strictly smaller" with no infinite descent.

A relation $\prec$ on a set A is *well-founded* if there is no infinite chain

$$a_0 \succ a_1 \succ a_2 \succ \cdots$$

with each step strictly decreasing.

Typical examples:

- $(\mathbb{N}, <)$ is well-founded.
- Lexicographic orders on tuples of naturals are well-founded.
- The empty relation is well-founded.

In total correctness, a variant decreases according to a well-founded relation, so the loop cannot continue forever.

Soundness: What Must Be Shown

Theorem:

$$\vdash \{P\}\ c\ \{Q\} \Rightarrow \models_{\mathsf{par}} \{P\}\ c\ \{Q\}.$$

Proof method: induction on the derivation of the Hoare proof.

Representative cases:

- Assign: use the definition of state update.
- Seq: use the semantic rule for sequencing.
- While: use induction on the big-step derivation of the loop.

Why Not Structural Induction?

At first glance, one might try to prove soundness by structural induction on commands.

That works for Assign, Skip, Seq, and If. But the While case is the problem:

- the loop body is executed an arbitrary number of times;
- the proof obligation is about *all* executions, not just the syntax of the command;
- the key induction step is on the *big-step derivation* of the loop, or equivalently on the number of iterations.
- the problem is that in the loop rule, the term hypothesis is as "big" as the on on the conclusion

So structural induction on commands is too shallow for termination and loop reasoning.

Soundness Sketch for While

Assume:

$$\models \{I \wedge b\}\ c\ \{I\}.$$

Need:

$$\models \{I\}\ \text{while } b \text{ do } c\ \{I \wedge \neg b\}.$$

If the program does not terminate, partial correctness is automatic.
Take $\langle \text{while } b \text{ do } c, \sigma\rangle \Downarrow \sigma'$ and $I(\sigma)$. Induct on this big-step derivation.

- If b is false initially, $\sigma' = \sigma$, so $I \wedge \neg b$ holds.
- If b is true, then the loop unfolds as

$$\langle c, \sigma\rangle \Downarrow \sigma_1 \qquad \text{and} \qquad \langle \text{while } b \text{ do } c, \sigma_1\rangle \Downarrow \sigma'.$$

  By the premise, $I(\sigma_1)$ after the body execution. The induction hypothesis applies to the remaining loop run from σ_1, so the final store σ' satisfies $I \wedge \neg b$.

Completeness: The Subtle Direction

Completeness asks:

$$\models \{P\}\ c\ \{Q\} \Rightarrow \vdash \{P\}\ c\ \{Q\}.$$

For expressive assertion languages, Hoare logic is relatively complete:

- complete relative to the ability to prove all true assertion-logic implications: whenever an implication $A \Rightarrow B$ is semantically valid, the assertion theory can derive it;
- arithmetic itself may be incomplete, so absolute completeness is too much to ask.

Strongest Postconditions and Completeness

One proof idea uses strongest postconditions:

$$\mathrm{sp}(P, c)(\sigma') \equiv \exists \sigma.\ P(\sigma) \wedge \langle c, \sigma \rangle \Downarrow \sigma'.$$

If the assertion language can express $\mathrm{sp}(P, c)$, then:

$$\models \{P\}\ c\ \{\mathrm{sp}(P, c)\}$$

is derivable by induction on c.

For any valid $\{P\}\ c\ \{Q\}$, we have:

$$\mathrm{sp}(P, c) \Rightarrow Q.$$

Then use Consequence.

Why sp Is Derivable: Assignment

Consider $c \equiv x := e$. Then the strongest postcondition is:

$$\mathrm{sp}(P, x := e)(\sigma') \equiv \exists \sigma.\ P(\sigma) \wedge \sigma' = \sigma[x \mapsto [\![e]\!]_\sigma].$$

This is exactly the postcondition produced by the assignment rule:

$$\{P[x \mapsto e]\}\ x := e\ \{P\} \qquad \text{Assign}$$

So for assignments, the derivation of $\models \{P\}\ x := e\ \{\mathrm{sp}(P, x := e)\}$ is immediate from the rule itself.

Why sp Is Derivable: While

Consider $c \equiv \text{while } b \text{ do } d$. The strongest postcondition is:

$$\text{sp}(P, c)(\sigma') \equiv \exists \sigma.\ P(\sigma) \wedge \langle \text{while } b \text{ do } d, \sigma \rangle \Downarrow \sigma'.$$

To derive $\models \{P\}\ c\ \{\text{sp}(P, c)\}$, reason by induction on the big-step derivation:

- If b is false, the loop stops immediately and the final state is the current one.
- If b is true, one body execution produces an intermediate state σ_1, and the induction hypothesis applies to the remaining loop from σ_1.
- The proof therefore mirrors the recursive structure of the semantics, not just the syntax of the command.

The Invariant Behind the While Case

For the completeness argument, the invariant is the reachable-state predicate itself:

$$I(\sigma) \equiv \mathrm{sp}(P, \text{while } b \text{ do } d)(\sigma).$$

Then:

- $P \Rightarrow I$, by unfolding the definition of sp.
- If b is true and d steps from σ to σ_1, then $I(\sigma_1)$ still holds, because σ_1 is reachable by one more loop unfolding.
- If b is false, the postcondition $I \wedge \neg b$ is exactly the strongest postcondition we want.

So the "invariant" in the completeness proof is semantic: it is the set of states reachable from the precondition.

What Completeness Does Not Give You

Completeness is not an automatic verifier.

It does not solve:

- inventing good loop invariants;
- making proofs short or readable;
- proving total correctness without well-founded termination arguments.

Its value is conceptual: the proof rules are not missing a program construct.

Weakest Liberal Preconditions

For partial correctness, the weakest liberal precondition:

$$\mathsf{wlp}(c, Q)(\sigma) \quad \text{iff} \quad \forall \sigma'.\ \langle c, \sigma \rangle \Downarrow \sigma' \Rightarrow Q(\sigma').$$

Then:

$$\models_{\mathsf{par}} \{P\}\ c\ \{Q\} \quad \text{iff} \quad P \Rightarrow \mathsf{wlp}(c, Q).$$

"Liberal" means nontermination is allowed.

Weakest Preconditions for Total Correctness

For total correctness:

$$\mathsf{wp}(c, Q)(\sigma) \quad \text{iff} \quad \exists \sigma'.\ \langle c, \sigma \rangle \Downarrow \sigma' \wedge Q(\sigma').$$

For deterministic IMP this captures termination and the desired final state.

Relationship:

$$\mathsf{wp}(c, Q) \Rightarrow \mathsf{wlp}(c, Q),$$

but not conversely.

Predicate Transformers for Basic Commands

$$
\begin{aligned}
\mathrm{wlp}(\mathrm{skip}, Q) &= Q \\
\mathrm{wlp}(x := a, Q) &= Q[x \mapsto a] \\
\mathrm{wlp}(c_1; c_2, Q) &= \mathrm{wlp}(c_1, \mathrm{wlp}(c_2, Q)) \\
\mathrm{wlp}(\text{if } b \text{ then } c_1 \text{ else } c_2, Q) &= (b \Rightarrow \mathrm{wlp}(c_1, Q)) \wedge (\neg b \Rightarrow \mathrm{wlp}(c_2, Q)).
\end{aligned}
$$

For loops, exact weakest preconditions are fixed points. In practice, invariants are human-supplied approximations.

WP Example

Compute:

$$\mathsf{wlp}(x := x + 1;\ y := x,\ y = x \wedge x > 0).$$

$$\begin{aligned}
&\mathsf{wlp}(x := x + 1,\ \mathsf{wlp}(y := x,\ y = x \wedge x > 0)) \\
&= \mathsf{wlp}(x := x + 1,\ x = x \wedge x > 0) \\
&= x + 1 = x + 1 \wedge x + 1 > 0 \\
&= x > -1.
\end{aligned}$$

Predicate transformers turn verification into calculation.

Assert and Assume

Two auxiliary commands are useful for verification condition generation:

- assert b: check b at runtime.
- assume b: proceed only under the assumption that b holds.

Their proof roles are different:

$$\{P \wedge b\}\ \text{assert}\ b\ \{P \wedge b\} \qquad \{P\}\ \text{assume}\ b\ \{P \wedge b\}.$$

Intuition:

- assert generates a proof obligation.
- assume strengthens the context for what follows.

Modular Verification

Modular verification splits a program into smaller proofs with intermediate contracts.

$$\frac{\{P\}\ c_1; \text{assert } P'; \text{assume } Q'; c_3\ \{Q\} \wedge \{true\}\ \text{assume } P'; c_2;\ \text{assert } Q'\ \{true\}}{\{P\}\ c_1; c_2; c_3\ \{Q\}}$$

Nondeterministic Choice

Some languages allow a command to choose among multiple possible next steps.

Write $c_1 \sqcup c_2$ for nondeterministic choice. Operationally, either branch may be taken:

$$\langle c_1, \sigma \rangle \Downarrow \sigma' \Rightarrow \langle c_1 \sqcup c_2, \sigma \rangle \Downarrow \sigma' \qquad \langle c_2, \sigma \rangle \Downarrow \sigma' \Rightarrow \langle c_1 \sqcup c_2, \sigma \rangle \Downarrow \sigma'.$$

This is adversarial: the proof must work for every possible branch.

Extending Triples to Nondeterminism

With nondeterministic commands, we keep the same contract shape but extend the meaning:

$$\models_{\mathsf{par}} \{P\}\ c\ \{Q\} \quad \text{iff} \quad \forall \sigma, \sigma'.\ P(\sigma) \land \langle c, \sigma \rangle \Downarrow \sigma' \Rightarrow Q(\sigma').$$

Now the big-step judgment is read as a relation, so a program may have several possible final states. For partial correctness, all of them must satisfy the postcondition.

Total Correctness with Nondeterminism

Total correctness becomes stronger:

$\models_{\text{tot}} \{P\}\ c\ \{Q\}$ iff $\forall \sigma.\ P(\sigma) \Rightarrow$ every possible execution from σ terminates in a Q-state.

So nondeterminism changes the termination claim too:

- Problem is, our big step semantics is not well equipped to model programs that may terminate on some executions diverge on others
- small step semantics
- divergence relations
- ...

Hoare Rule for Choice

The rule for demonic choice is:

$$\frac{\{P\}\ c_1\ \{Q\} \quad \{P\}\ c_2\ \{Q\}}{\{P\}\ c_1 \sqcup c_2\ \{Q\}}\ \text{Choice}$$

Why both premises?

- the execution may resolve to either branch;
- the postcondition must survive whichever branch is chosen;
- so the same precondition must be strong enough for both.

Example: Increment by 1 or 2

Consider:

$$x := x + 1 \sqcup x := x + 2.$$

We can prove:

$$\{x \geq 0\}\ x := x + 1 \sqcup x := x + 2\ \{x > 0\}.$$

Reason:

- after $x := x + 1$, we have $x > 0$;
- after $x := x + 2$, we also have $x > 0$;
- therefore the choice command satisfies the same postcondition.

Suggested Reading

- C. A. R. Hoare, "An Axiomatic Basis for Computer Programming", *CACM*, 1969.
- Robert W. Floyd, "Assigning Meanings to Programs", 1967.
- Glynn Winskel, *The Formal Semantics of Programming Languages*, chapters on operational semantics and Hoare logic.
- Hanne Riis Nielson and Flemming Nielson, *Semantics with Applications*, sections on IMP and axiomatic semantics.
- Mike Gordon, *Programming Language Theory and its Implementation*, Hoare logic chapters.
- Tobias Nipkow and Gerwin Klein, *Concrete Semantics*, chapters on IMP, Hoare logic, and verification condition generation.

Optional Advanced Reading

- Stephen A. Cook, "Soundness and Completeness of an Axiom System for Program Verification", 1978.
- Edsger W. Dijkstra, *A Discipline of Programming*, weakest preconditions.
- David Gries, *The Science of Programming*, predicate transformer style.
- Apt, Olderog, and de Boer, *Verification of Sequential and Concurrent Programs*.

Takeaways

- Hoare triples express local contracts for commands.
- Partial correctness says results are right if the program terminates.
- Total correctness adds termination, usually via variants.
- Loop invariants are the main creative step.
- Soundness connects proof rules to semantics.
- Relative completeness says the rules are expressive enough, assuming the assertion logic is.
- Weakest preconditions recast verification as predicate calculation.

Tools Using Hoare Logic

Many verification tools turn Hoare-style proofs into generated proof obligations.

Typical workflow:

- write contracts and loop invariants;
- compute verification conditions with weakest preconditions;
- discharge the resulting assertions with an SMT solver or theorem prover.

Examples include Dafny, Why3, Frama-C/WP, and SPARK.

The common idea is simple: Hoare logic provides the specification layer, and automation handles the arithmetic and branching details.